腾讯Ray团队实践：K8s + Ray如何支撑超大规模AI Workload

腾讯技术工程 Jul 13, 2026, 2:36 AM 广东 18 listened

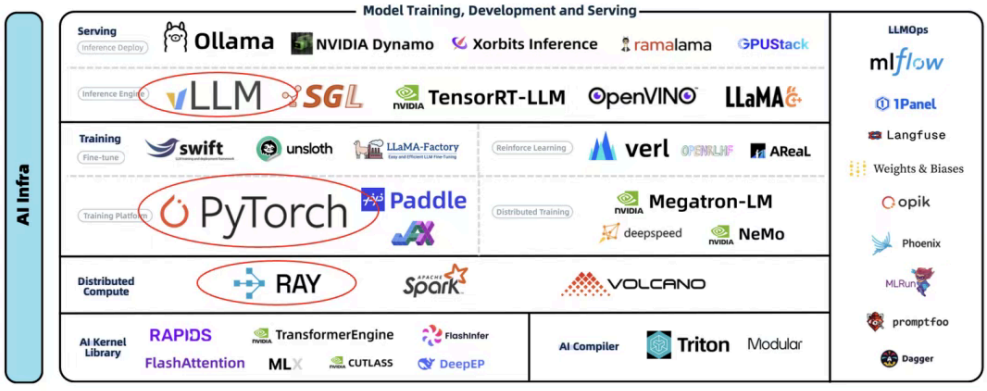


作者：charliecli

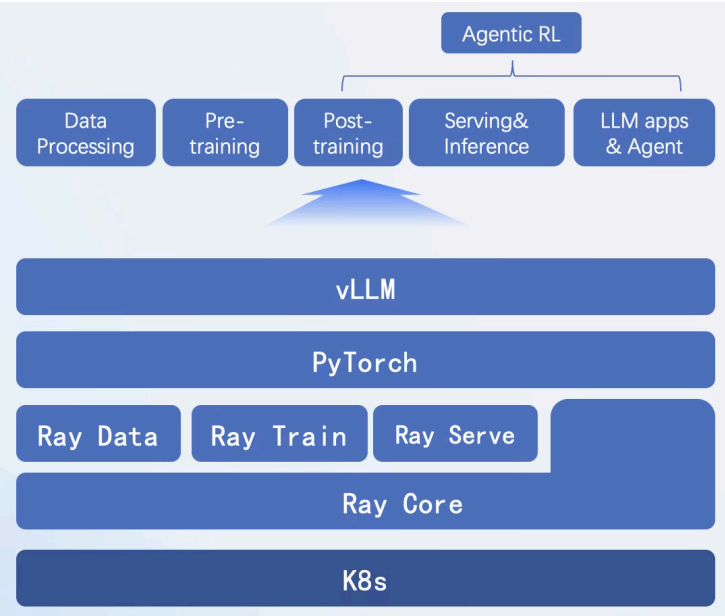
随着大模型时代的到来，AI 基础设施（AI Infra）正在经历深刻的变革。面对日益复杂的计算需求，传统上与单一计算范式深度耦合的调度系统已难以应对全局性挑战。本文结合开源社区的演进趋势与工业界的超大规模落地实践，深入探讨 Ray 的技术定位与核心设计逻辑，并阐述它如何与 Kubernetes（以下简称 K8s）进行协同设计（co-design），共同构建大模型时代 AI Workload 调度的通用范式。 注：文章的图片内容来源于腾讯 Ray 团队在 Qcon 全球软件开发大会的分享：
https://qcon.infoq.cn/2026/beijing/presentation/7002

一、 大模型时代 AI 基础设施的技术栈演进

要理解这一全新的调度范式，首先需要审视当前大模型基础设施的技术栈现状。借助蚂蚁开源技术委员会绘制的 AI Infra 开源生态全景图 [1]，我们可以全面了解当下 AI Infra 领域的主要开源项目。



从这张全景图中可以提炼出一条典型的 AI Infra 技术栈：**Ray + PyTorch + vLLM**。值得一提的是，这三个项目目前均隶属于 PyTorch 基金会（其中 Ray 于 2025 年 PyTorch 大会上正式作为托管项目加入）。在此之上，再叠加工业界事实标准的部署与调度底座 **K8s**，便构成了 **K8s + Ray + PyTorch + vLLM** 的黄金组合。这套技术栈贯穿大模型生命周期的全链路，涵盖数据处理、预训练、后训练、在线推理与 Agent 等场景。

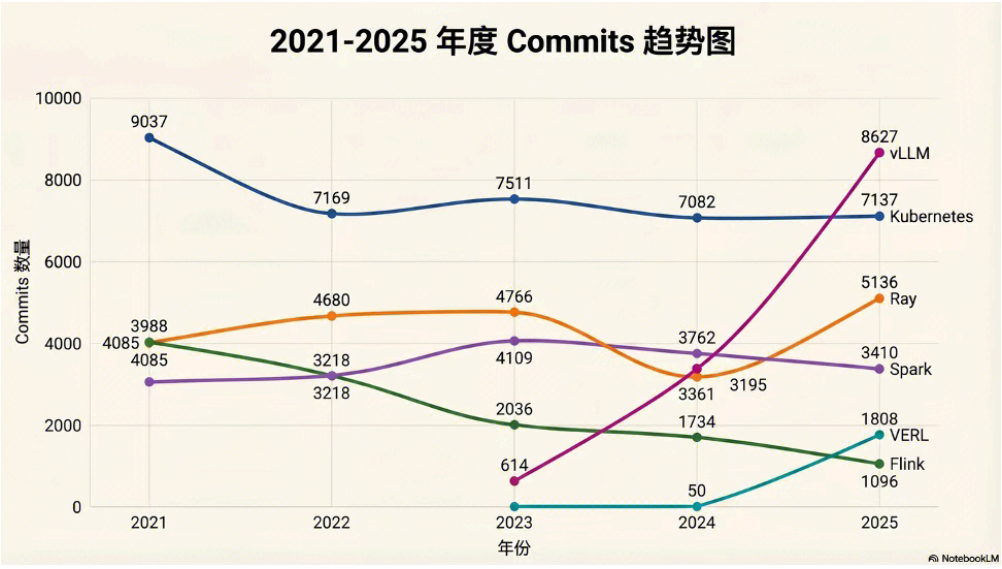


为更直观地理解这套技术栈的运作方式，我们以强化学习（RLHF）为例。当前主流的 RL 训练框架普遍采用"**训推分离**"架构：

- **训练端**：依托 PyTorch 生态（如 Megatron、DeepSpeed）提供高性能训练能力。
- **推理端**：以 vLLM 作为核心推理后端（Backend）。
- **编排与调度**：由 Ray 串联全局，承担训推流程的编排以及角色间的复杂通信。
- **底层基座**：K8s 作为应用部署的事实标准，提供底层物理资源支撑。

目前，业界 90% 以上的 RL 训练框架均构建于这套 K8s + Ray + PyTorch + vLLM 黄金组合之上。关于该组合的深度探讨，可参见四个项目的技术负责人在 Ray Summit 2025 上的对谈 [2]，本文不再赘述。

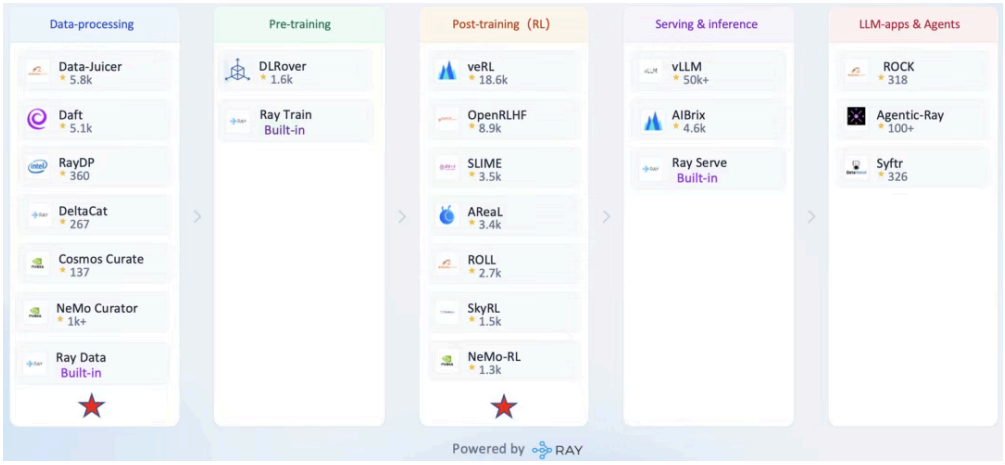
这套技术栈也已经过开源社区的真实检验。从 2021—2025 年的开源活跃度（以 Commit 数为指标）来看：



作为 AI 应用时代最关键的推理引擎，vLLM 在过去一年贡献了超过 8000 个 Commit，活跃度极高；Kubernetes 始终保持极高且稳定的活跃度，与其云原生部署事实标准的地位相

称；而 Ray 作为通用计算引擎，活跃度已明显超越 Spark、Flink 等传统大数据计算引擎。

让 Ray 在近两年迎来爆发的，正是它在两类核心场景中的不可替代性：**多模态数据处理** 与 **后训练/强化学习**。下图梳理了基于 Ray 构建的 AI Infra 开源项目，可以看出 Ray 在 **数据处理** 与 **后训练/强化学习** 这两个方向上的生态最为活跃。



此外，我们也汇总了 Ray 在国内主要企业的落地情况：



如今，国内头部厂商（包括 DeepSeek、月之暗面等）在多模态数据处理上几乎全面采用 Ray；90% 以上的 RL 训练框架基于 Ray 构建；主流云厂商均已提供 Ray 托管服务；阿里等企业也开始探索基于 Ray 构建 Agent Sandbox。在非 AI 场景方面，蚂蚁集团早在 2017—2018 年便已将 Ray 应用于图计算与隐私计算。

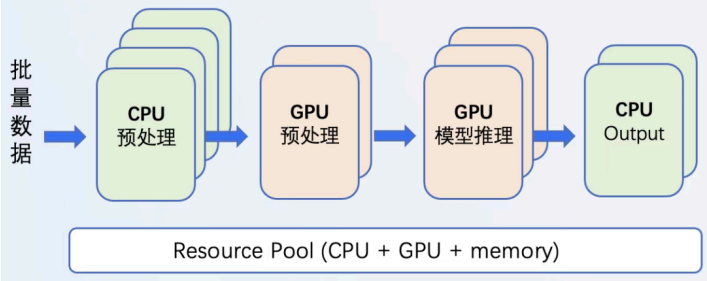
接下来，我们将从 AI Workload 调度的视角，深入解析大模型时代的 AI Infra 为何选择 Ray，以及 Ray 究竟解决了哪些传统计算引擎难以应对的调度痛点。

二、 基于 Ray 的 AI Workload 调度

我们首先通过两个典型场景，归纳当下 AI Workload 提出的调度需求。

1. 多模态数据处理

下图展示了多模态数据处理的典型 Pipeline：系统需要持续读取、处理并输出大批量多模态数据。整条 Pipeline 由多个 Stage 串联组成，其中既包含 CPU 密集型算子（如抽帧、格式转换），也包含强依赖 GPU 的算子（如 OCR、语音识别、大语言模型推理）。所有算子需要在统一的资源池内参与调度。

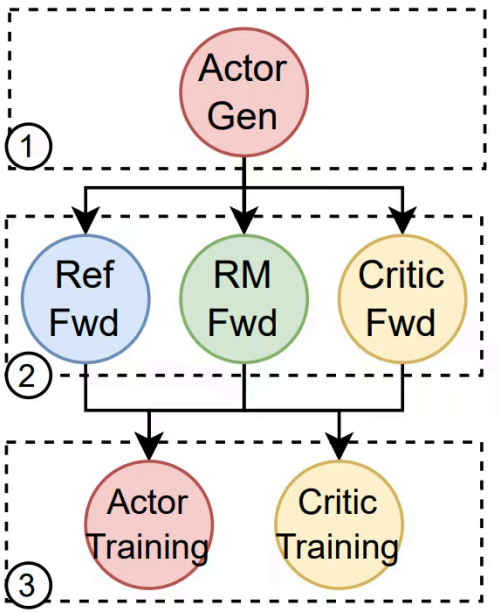


这带来了三大挑战：

- **异构调度**：CPU 与 GPU 算子需要被高效地匹配到对应的异构节点；
- **动态分配**：需根据实时负载动态调整各 Stage 的资源量与并发度，以打破吞吐瓶颈；
- **高容错**：由于链路长、耗时久，单点故障（如 OOM、GPU Error、Spot Instance 回收）几乎不可避免。因此容错粒度必须下探到 Stage、Pod 乃至进程级，避免单点故障拖垮整条 Pipeline。

2. 强化学习（RLHF）

以 RLHF 中一次典型的 PPO Training Step 为例：用户的 Prompt 经 Actor 推理生成 Response 后，需并发分发给 Reference、Reward 与 Critic 进行打分评估，再经 Advantage 计算回传给 Actor 与 Critic 完成参数更新。



这本质上是一个**多异构角色的协同调度问题**。它不仅涉及多种角色（其中 Actor 还可能进一步拆分为训练与推理两部分），而且各角色依赖的运行时完全不同（例如推理需拉起 vLLM/SGLang，训练需加载 FSDP/Megatron），并发设置与资源需求也各不相同。更复

杂的是，任务流转并非简单的线性 DAG，而是需要在多角色之间以复杂的多播（multicast）形式传递。

面对后训练/RLHF 中"异构、多角色协同"的调度需求，预训练阶段长期沿用的计算范式变得难以匹配。主流预训练框架（如 FSDP、Megatron）通常采用 **Multi-Controller / SPMD（Single Program, Multiple Data）** 范式，要求每个计算单元运行同构进程，并依赖同步 Barrier 与集合通信。这种范式存在三方面局限：灵活性差（难以表达异构角色）、容错率低（单点故障会导致整个通信组崩溃）、缺少一个能够以全局视角统一编排复杂任务流的中心角色。

为突破这一限制，大模型时代的强化学习框架（如 veRL[3]、SkyRL[4]）纷纷转向 **Single-Controller / MPMD（Multiple Program, Multiple Data）** 范式：引入一个中心 Driver（Single-Controller）来统一编排多个异构角色。Driver 能够以全局视角组织跨角色的复杂任务流；各异构角色内部仍可保留 SPMD 架构以获取局部高性能。借助中心 Driver，异构角色之间得以松耦合，容错也能在角色粒度上独立完成。

3. AI Workload 调度需求小结

基于上述两个典型场景，我们将大模型时代 AI Workload 的调度需求归纳为四点：

- **异构资源**：将异构算子/角色高效地调度到异构节点上
- **动态分配**：根据实时负载，为每个算子/角色动态分配计算资源，避免出现局部吞吐瓶颈
- **高容错**：局部故障不影响全局任务，出错的计算单元支持自动重新调度并恢复状态
- **原生支持 Single-Controller**：由 Single-Controller 统一编排跨角色/跨算子的复杂任务流

4. Ray 核心 API 设计

本节通过 Ray 的核心 API，展示上述四项调度需求如何在 Ray 中被一一满足。下面的 Python 代码片段简要模拟了 RLHF 训推分离场景：

```
import ray

def main():
    # 主函数中声明当前进程为中心 Driver
    ray.init()

    # 定义一个 Rollout 角色（类），该角色的每个实例需要：
    # 1. 分配 2 个 CPU 和 1 个 GPU
    # 2. 出错后无限次自动重启
    @ray.remote(num_cpus=2, num_gpus=1, max_restarts=-1)
    class RolloutWorker:
        def __init__(self):
            # 如果当前实例为重启状态（非首次创建）
            if ray.get_runtime_context().was_current_actor_reconstructed:
                # 自定义状态恢复行为
                self._recover_state()

        def generate(self, prompt):
            return"Hi there"
```

```python
# 定义一个 Trainer 角色（类），该角色的每个实例需要分配 2 个 GPU
@ray.remote(num_gpus=2)
class Trainer:
    def fit(self, experience):
        return 0

# 创建 2 个 RolloutWorker 远程实例，组成 rollout worker group
rollout_worker_group = [RolloutWorker.remote() for _ in range(2)]

# 创建 1 个 Trainer 远程实例
trainer = Trainer.remote()

# 开始 RL 训练流程
while True:
    for rollout_worker in rollout_worker_group:
        # 远程调用（异步）RolloutWorker 实例的 generate 方法
        response = rollout_worker.generate.remote("Hello")
        # 将 RolloutWorker 实例的 response 派发给 Trainer 实例
        ret = trainer.fit.remote(response)

    # 动态增加远程实例（提升 rollout 并发度）
    rollout_worker_group.append(RolloutWorker.remote())
```

在上述代码中，我们首先通过 `ray.init` 将当前进程声明为中心 Driver（**Single-Controller**）。然后通过 `@ray.remote` 装饰器定义**异构**角色（类定义），并声明每个角色实例的**资源**与**容错**需求。对每个角色，我们都可以通过 `remote()` 接口创建任意数量的远程实例（进程），并按需分组管理。在训练循环中，可以调用任意远程实例的方法，并将上游返回值（`ObjectRef`）派发给指定下游实例，从而编排任意形式的任务流。运行过程中也可以**动态**增减角色实例，以调节各角色的并发度。

本质上，Ray 作为分布式计算引擎，其调度的核心对象是"进程级计算单元"。在进程粒度上，Ray 同时实现了 **异构资源调度**、**动态分配** 与 **高容错** 能力。中心 Driver（**Single-Controller**）则可充分利用这些调度能力，灵活编排任务流。

5. 分布式计算引擎对比：调度能力

总结完 AI Workload 的调度需求与 Ray 的对应支持之后，我们进一步将 Ray 与其他主流分布式计算引擎（Spark、Flink、PyTorch）在调度能力上进行对比：

	Spark	**Flink**	**PyTorch**	**Ray**
计算范式	BSP（批处理）	流式 Dataflow	SPMD	无范式（通用分布式）
异构资源	粗粒度（Stage 内同构）	粗粒度（Slot 切分）	不支持	细粒度（进程级）
动态分配	粗粒度（AQE，Stage 间生效）	支持	不支持（静态通信组）	细粒度（进程级）

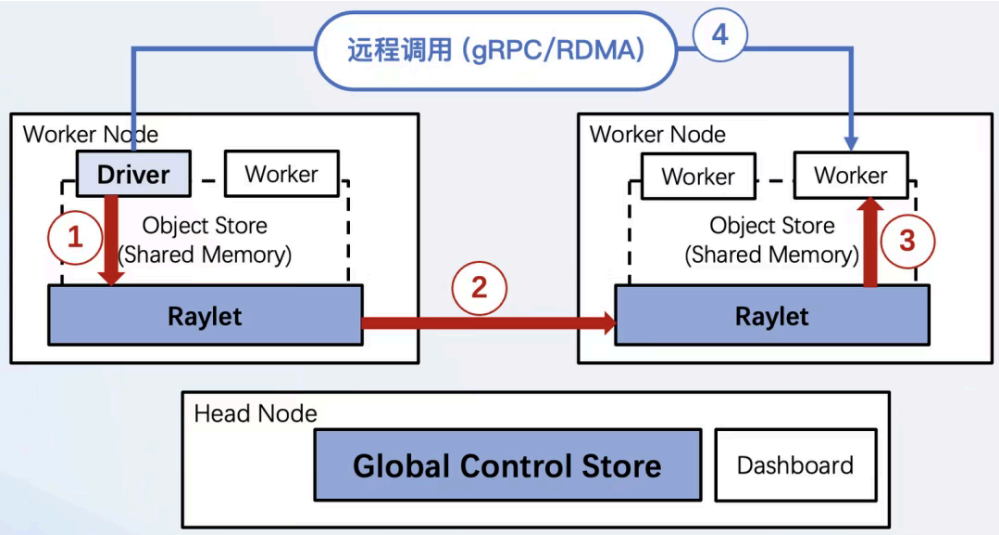
	Spark	Flink	PyTorch	Ray
容错	粗粒度（RDD Lineage 重算）	粗粒度（Checkpoint 回滚）	粗粒度（整组重启）	细粒度（进程级）
Single-Controller	支持	支持	不支持	支持

从计算范式来看，Spark 绑定了 BSP 批处理，Flink 绑定了流式 Dataflow，PyTorch 绑定了 SPMD；而 Ray 本身是无范式的——基于其提供的进程级计算单元，用户可以根据业务形态自由构建任意计算范式。反过来看，正因为 Spark、Flink、PyTorch 与固定计算范式深度耦合，它们在 **异构资源调度**、**动态分配** 与 **容错能力** 方面均缺乏足够细粒度的支持，难以全面覆盖大模型时代 AI Workload 的多样化需求。

综合来看，Ray 凭借进程级调度的灵活性，成为复杂 AI Workload 调度的最优解。

6. Ray 架构与调度实现

在本章最后，我们简要介绍一下 Ray 的整体架构与调度实现。



Ray 集群架构如上图所示。其中，Head 节点上运行 Global Control Store（GCS），负责集群元数据管理与节点状态同步；每个 Worker 节点上运行 Raylet，负责调度决策与本地进程管理。当用户的中心 Driver 创建角色实例时，调度会经历以下流程（实际略有差异，此处仅作示意）：

1. 调度请求首先发送至本地 Raylet，由其做出调度决策，选择目标节点
2. 调度请求被转发至目标节点的 Raylet
3. 目标节点 Raylet 根据本地资源状态决定是否接受请求；若接受，则在本地创建 Worker 进程并运行角色实例
4. 调度完成后，Driver 即可通过 gRPC（或 RDMA）直接调用远程实例的方法。

目前，Raylet 内置了多种调度策略，包括 Round-Robin 调度、按堆叠水位调度、Node/Label 亲和性调度、Gang Scheduling 以及 Data Locality 调度，用户可以根据业务场景的调度需求自由调配。对更多技术细节感兴趣的读者可以参考 Ray 官方文档 [5]。

三、 K8s + Ray 的协同调度范式

目前，Ray 的主流生产部署方式都构建在 K8s 之上。在近几届 Ray Summit 全球峰会上，AWS、Microsoft、Google 等主流云厂商也相继披露了各自基于 K8s + Ray 的部署方案。

为什么必须引入 K8s?

K8s 作为企业级基础设施，能够高效、统一地管理大规模物理资源池，并提供容器化部署、服务发现、存储编排，以及成熟的监控、运维与权限体系。借助 K8s 的成熟生态，Ray 才能快速在生产环境中实现稳定、规模化的落地。

K8s & Ray：职责分工

当采用 K8s 结合 Ray 的部署方式时，两者的职责分工如下所示：

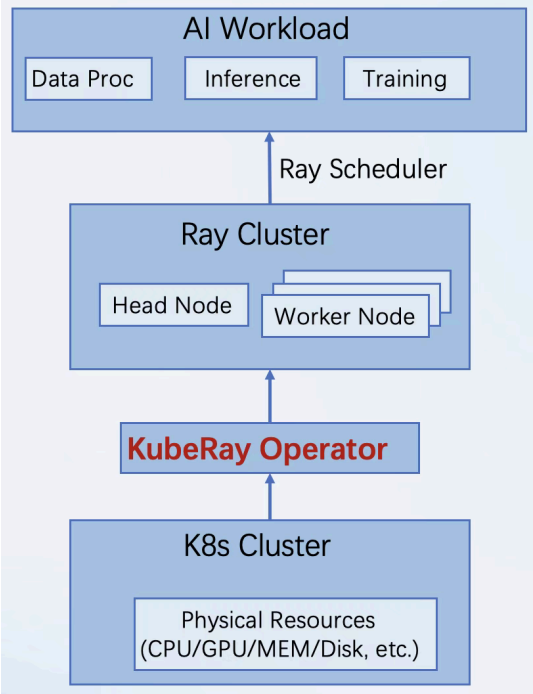
	K8s	**Ray**
定位	物理资源调度与管理	应用层调度与编排
资源管理对象	大规模物理节点资源	Ray Cluster 内的资源
生命周期管理	容器/Pod	Worker 进程
调度对象	Pod → 物理节点	Worker 进程 → Ray Node (Pod)
API 形态	YAML 声明式	Python 编程式

从定位上来说，K8s 是物理资源调度与管理，而 Ray 在其之上充当应用层调度与编排。从调度上来说，K8s 负责将 Pod 调度到物理节点上，整个流程涉及 API Server、Scheduler、Kubelet 等多个组件以及状态同步/持久化操作；而 Ray 则以二层调度的方式，负责将 Worker 进程调度到 Pod（Ray Node）上，调度策略更加贴合上层任务的需求，整体流程更加轻量。从 API 设计上来说，K8s 是 YAML 声明式的，更面向集群管理/运维人员；而 Ray 提供编程式接口，更适合分布式应用的研发人员。

K8s & Ray：协同调度

将 K8s 与 Ray 结合做协同调度时，整体流程可以概括为：K8s 负责将资源从统一资源池中分配给 Ray；而 Ray 负责将分配到的资源以更细粒度分配给上层任务。

为了将 Ray 无缝接入 K8s 生态，Ray 开源社区提供了 KubeRay Operator。用户提交自定义的 `RayCluster` CR 后，由 KubeRay Operator 负责持续调谐，创建并维护对应的 Ray 集群。Ray 集群包含 Head 节点和任意数量、异构规格的 Worker 节点，并支持运行时的自动扩缩容。Ray 集群内的资源则进一步由 Ray Scheduler 以轻量、细粒度的方式分配给上层任务。



如上图所示，通过 K8s → KubeRay → Ray Scheduler 这条自底向上、由粗到细的协同调度链路，资源可以被稳定、高效、灵活地分配给 AI 任务。

四、K8s + Ray 在腾讯的落地实践

从开源社区的视角来看，K8s → KubeRay → Ray 这套协同调度方案已经相当成熟。然而，当我们将它真正落地到腾讯内部，尤其是面对企业级的超大规模集群时，现实远比想象复杂。

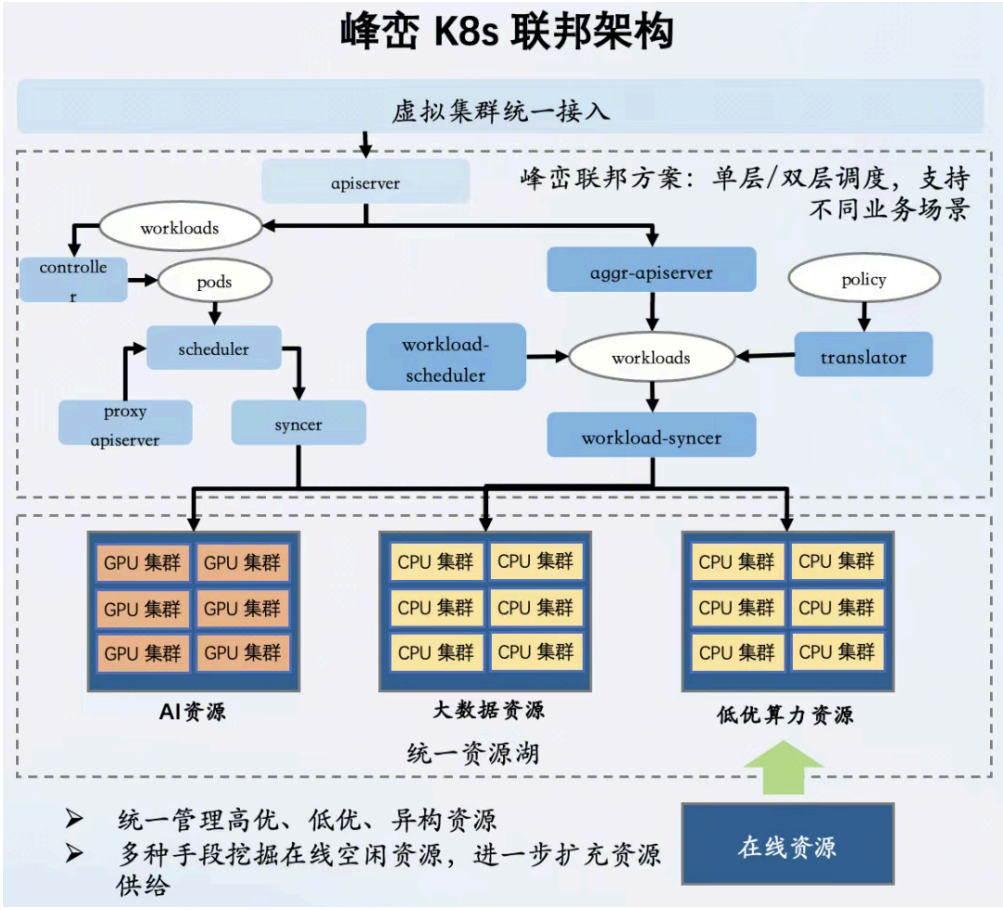


在腾讯 TEG 内部，Ray 平台的整体架构涉及多个层级。最底层是峰峦 K8s 云原生架构，作为统一算力平台管理海量异构算力资源，并提供联邦调度、智能调度等能力。中间层涵盖完整的 Ray 生态：从 Ray 调度内核，到数据处理、训练、在线服务三大高阶框架能力，再到 DAGFlow 离在线一体化的服务封装。上层是数据平台与机器学习平台，承接各业务线的具体需求。

首先，我们要面对的复杂性来自底层峰峦 K8s 云原生架构。这套架构有几个关键特点：

- K8s 物理集群不直接向上暴露，由联邦层进行统一管理

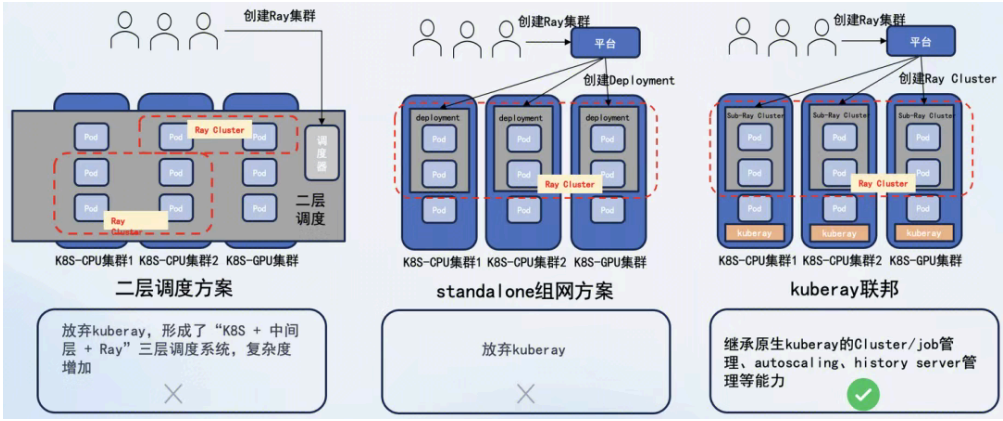
- 生产环境存在上百个 K8s 物理集群，以 CPU 算力与 GPU 算力维度分离
- WeData 数据平台与太极 AI 平台分别维护两套独立的 K8s 基础设施



这些特点给 Ray 的落地带来了显著挑战：Ray 提供的是异构融合计算能力，需要 K8s 云原生层供给"CPU + GPU"混合资源，而这些资源往往分散在不同的 K8s 物理集群中；但社区版 KubeRay 仅支持 RayCluster 在单一 K8s 物理集群内部署与调度。

1. 核心挑战：云原生联邦架构的演进

针对底层 K8s 架构现状，我们首要的目标是支持 RayCluster 跨 K8s 集群部署。在技术选型阶段，我们对以下三种方案进行了深入评估：

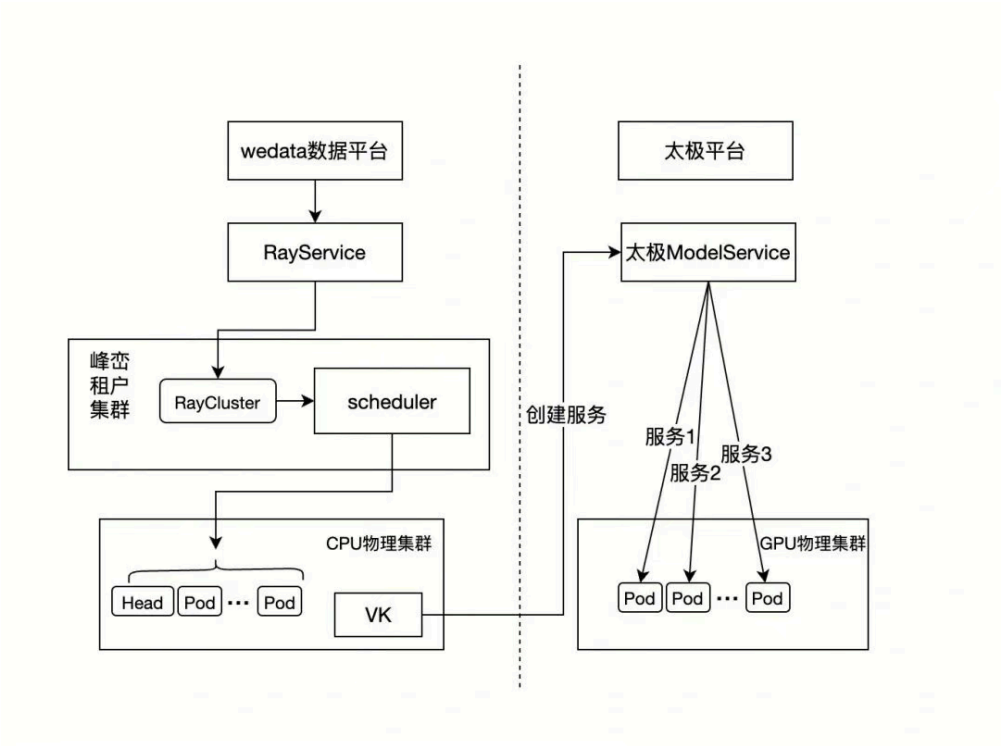


1. **二层调度方案**：在各 K8s 集群中预申请大量 Pod 常驻，并在这些 Pod 之上构建一层自定义调度器来调度 RayCluster。该方案引入了额外的调度层，使整个系统从"K8s + Ray"的两层调度演变为"K8s + 中间层 + Ray"的三层调度，显著增加了系统复杂度与调优难度。

2. **平台层 Standalone 组网方案**：在平台侧直接调度多个 K8s 集群的 Deployment 形成多个 Pod 组，并在每个 Pod 内部启动 Ray 进程，指定其中一个节点为 Head，以 Standalone 方式完成组网。该方案虽然没有引入额外的调度层，但放弃了 KubeRay 的能力，需要在平台层重新构建大量替代逻辑。
3. **KubeRay 联邦方案**：保留原生 KubeRay，并在其之上扩展支持跨集群联邦。

我们希望尽可能地兼容并复用开源社区的已有能力，因此最终选择了方案 3——扩展 KubeRay 以支持集群联邦。在落地过程中，KubeRay 联邦架构经历了两个演进阶段：

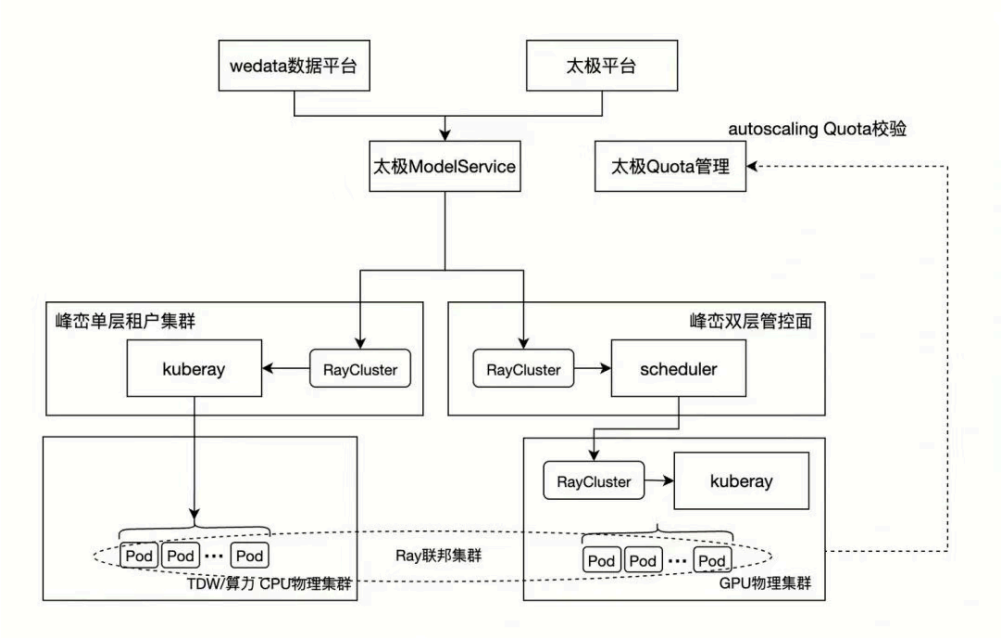
阶段一：Virtual Kubelet（VK）架构



为实现跨集群组网，我们构建了第一版基于 Virtual Kubelet（VK）的架构：在 CPU 集群上保留完整的 KubeRay 能力，并通过 VK 将太极集群的 GPU 资源抽象为虚拟 Kubelet。创建 GPU Pod 的实际链路是：KubeRay 调用 VK，VK 再调用 ModelService 创建太极服务，最终由太极服务拉起真正的 Pod。该方案在 TEG 内部成功落地，但在规模化推广时暴露出明显瓶颈：架构依赖倒置（底层 VK 反向依赖上层服务），调用链路冗长——每创建一个 GPU Pod 都要触发一次太极服务创建。在后期的业务落地中，当 RayCluster 节点数超过 100 时，太极链路便会承受较大压力，难以满足生产需求。

阶段二：KubeRay 联邦架构

考虑到 VK 架构的瓶颈，我们进一步推进了更合理的 KubeRay 联邦架构。该架构在多个 K8s 物理集群中并发部署 KubeRay Workload，并通过配置约束仅由其中一个 Workload 启动 GCS（Global Control Store），其余 Workload 的节点均作为 Worker 节点组加入同一 Ray 集群。在此基础上，我们还实现了联邦集群的 Autoscaling 能力，借助联邦机制完成全局资源弹性。

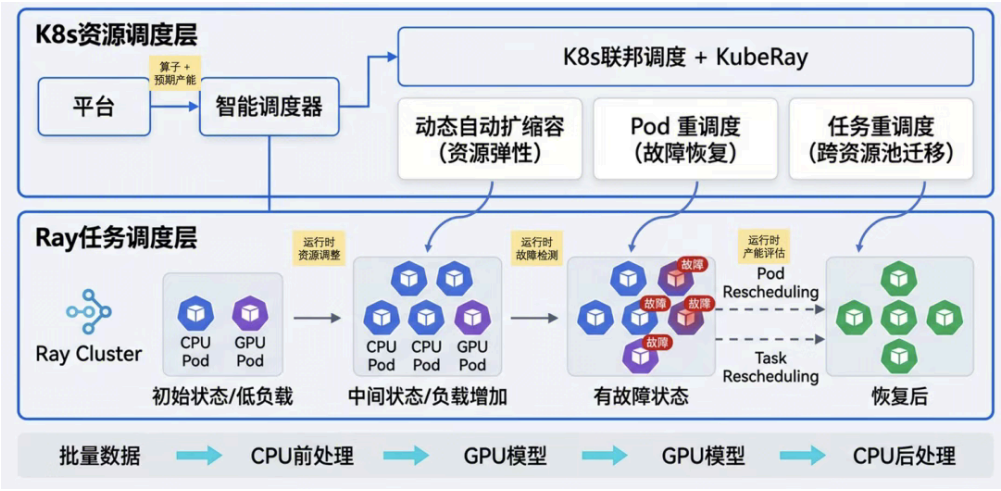


完成该改造后，我们实现了异构资源调度的统一——CPU 与 GPU 算力可以通过同一个入口灵活组网，单集群规模可达万卡以上；同时 Wedata 数据平台与太极 AI 平台得以复用相同的调度链路，达成了 Data + AI 的调度统一。

2. 跨层协同设计：跨层弹性调度

解决了基础的跨集群组网问题后，我们开始直面业务在规模化推广中遇到的真实痛点。在多模态数据处理场景中，我们发现业务方往往难以根据产能要求直接评估出所需的 CPU/GPU 规格与数量；并且在任务运行前也无法准确预知瓶颈所在（例如华南 CPU 与华北 GPU 跨地域协同导致的 IO 带宽瓶颈，或 GPU 节点上 CPU 预处理资源不足导致 GPU 利用率低下），使得运行时效率难以保障。

为此，我们设计了**跨层弹性调度**机制：业务方无需再纠结资源规格，只需提交"算子"与"预期产能"，由 Ray 在运行时动态完成资源匹配。



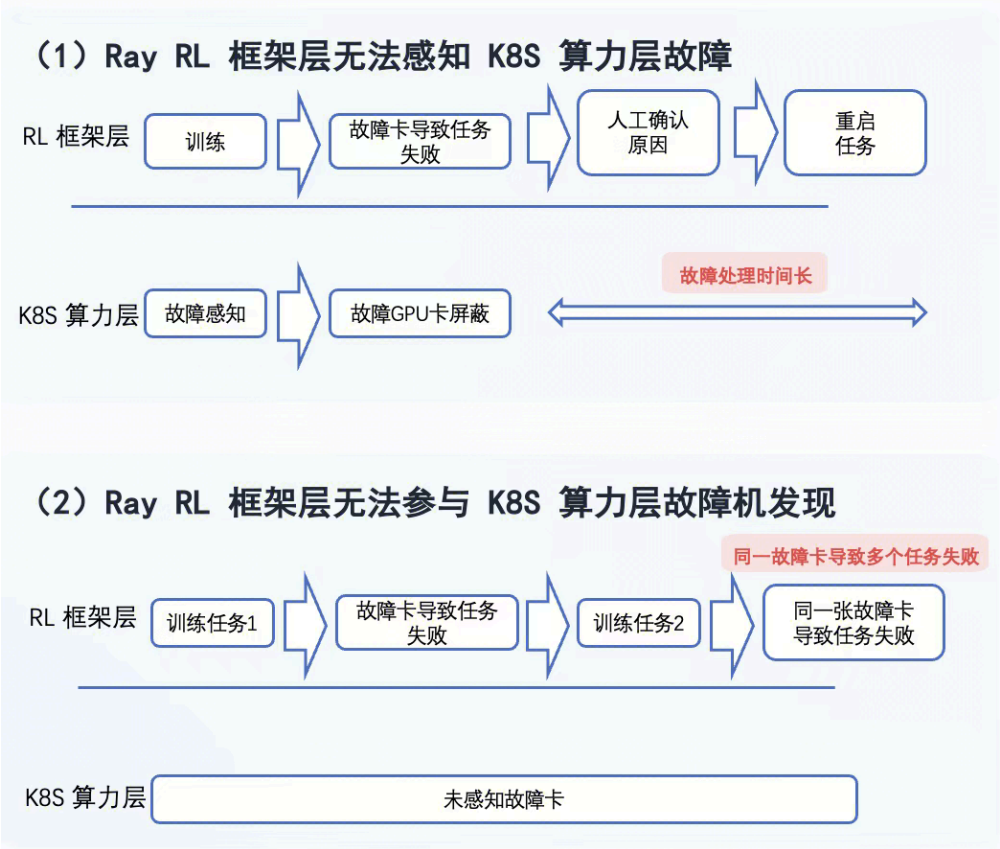
整体流程如下：用户算子和预期产能通过业务平台提交后，由智能调度器输出推荐配置并下发至 K8s，KubeRay Operator 通过跨 K8s 联邦调度创建出初始 Ray 集群。任务运行过程中，系统会进行三级自动调优：

1. **动态扩缩容**：根据 Ray 集群资源水位与任务负载情况，实时调整异构 Ray 节点（Pod）的数量，避免长时间资源空转或数据堆积。
2. **Pod 重调度**：对出现隐患（例如 GPU ECC Error、磁盘空间不足等）的 Ray 节点触发重新调度。
3. **任务重调度**：若上述两级调整仍无法达到预期产能，则对整个任务进行重新调度，由 K8s 层重新做智能资源池决策后任务继续执行。

每一级自动调优都需要 Ray 任务调度层与 K8s 资源调度层协同处理。通过这一系列闭环反馈，系统可自动将任务调整至最佳运行状态。

3. 跨层协同设计：跨层自动化容灾

除了资源配置，稳定性是业务的另一大需求。这里我们以对故障高度敏感的强化学习场景为例。

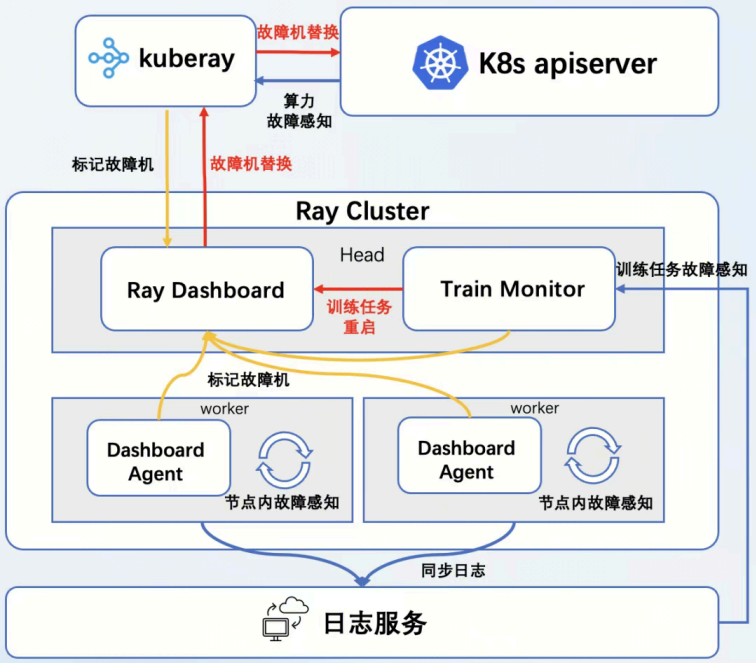


在现有的强化学习业务中，RL 框架层与 K8s 算力层之间缺乏故障协同处理，主要表现为两类问题：

1. 当 K8s 算力层检测并屏蔽故障卡后，RL 框架无法及时感知，往往要等到训练任务失败、并经人工确认后才能介入处理。
2. RL 训练任务因故障卡失败后，无法自动通知 K8s 算力层做调度屏蔽，导致任务再次拉起时可能再次命中同一张故障卡，反复失败。

为此，我们打造了**跨层自动化容灾**方案，覆盖**故障感知**、**故障标记**与**故障处理**三个环节。

- **全方位故障感知**：在 Ray Worker 节点内，由 Dashboard Agent 周期性检测关键资源指标（如磁盘、GPU 健康度等）；在 Head 节点内，由 Train Monitor 基于训练任务日志感知异常；在 K8s 算力层，由定期巡检发现故障卡。
- **统一故障标记**：任意组件感知到故障后，都可通过 Ray Dashboard 提供的开放接口对故障节点进行统一标记。
- **故障节点替换与任务续训**：当训练任务无法推进而触发重启时，所有被标记的故障节点会被自动替换，并在 K8s 层完成故障卡屏蔽；新任务拉起时将自动避开所有已知故障节点。



通过这一闭环机制，故障从"被动等待人工介入"转变为"自动感知—自动隔离—自动恢复"，显著提升了大规模 RL 训练任务的稳定性。

五、 未来展望

通过上述跨层协同调度的实践，我们打通了 K8s 与 Ray，沉淀出一套行之有效的 AI Workload 调度通用范式。面向未来，我们将在以下三个方向持续深耕：

1. **更原生的 Ray 联邦架构**：当前的 KubeRay 联邦方案仍需要接入层感知，在不同场景间部署仍需迁移成本；未来希望将这部分能力收敛进 K8s + Ray 技术栈内部，进一步提升易用性与普适性。
2. **更通用的分布式底座**：面向多模态数据处理、预训练、强化学习、在线推理、Agent 应用等场景，构建更通用的分布式底座与平台，强化调度、通信、存储等通用分布式能力。
3. **更统一的 Agentic RL Infra**：在"训练 + 推理"统一到 Ray 计算范式的基础上，进一步覆盖 Agentic RL 场景中更广泛的调度统一问题，例如 Agent 与 Sandbox 运行环境的统一编排。

当 K8s 遇见 Ray，大模型时代的 AI Workload 调度正在开启新的篇章。期待更多开发者加入我们，共同见证并参与这场 AI 基础设施的变革。

References

[1] **Open Source LLM Development Landscape**

[2] **Ray Summit 2025 Keynote: AI OSS Stack Panel with vLLM + PyTorch + Kubernetes**

[3] **verl: Volcano Engine Reinforcement Learning for LLMs**

[4] **SkyRL: A Modular Full-stack RL Library for LLMs**

[5] **Ray Official Docs**

关于我们

我们是腾讯 Ray 团队， 为公司内的大数据和机器学习业务提供规模化的分布式调度与计算底座。 我们团队的优势是会结合“GPU 算力 + K8s + Ray + 训练/推理框架”多层能力进行 co-design， 并深度对接平台提供一站式的产品化能力。

团队隶属于腾讯 TEG 数据计算平台部，部门负责司内统一大数据和机器学习的数据智能融合平台建设，为大数据、机器学习业务提供全面高效的PaaS平台底座，围绕数据和算力，构建规模化的大数据和机器学习融合平台的全栈式服务能力。

我们正在招聘 Ray 分布式调度与计算工程师，感兴趣的朋友请通过 graysong@tencent.com 联系。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号



阅读 8709

留言

写留言

还没有留言，来写首评